

abstractizare

Șapte exerciții — `abstract` pentru familie, `interface` pentru capacități · TypeScript · Andreea

Unde lucrezi: `vanilla-ts/src/exercitii/` și `vanilla-ts/src/fise/`, peste ce ai făcut la fișele de moștenire și polimorfism.

Nou, ai voie: `abstract` pe clasă, pe metodă și pe câmp, `interface`, `implements`, `toString()` și `toJSON()`.

Încă NU folosim: `instanceof`, `any`, generice proprii, `static`, `enum`.

Cum se citește fișa: **Teorie** înainte, **Indiciu** după prima încercare, **leșirea așteptată** literal. Aici compilatorul verifică cel mai mult: la fiecare exercițiu rulezi întâi `npx tsc --noEmit` și citești ce spune, abia apoi `npx tsx`.

Partea 1 — clasa de sus nu se mai poate construi

1. Verificarea nu mai minte

nivel 1

Fișierul `05-verificari/main.ts`. Acum `Verificare.trece` întoarce `true` doar ca să compileze. Fă:

- `abstract class Verificare`.
- Metoda devine `abstract trece(valoare: string): boolean;` — fără corp, fără acolade, cu `;` la final.
- Scrie `class Contine extends Verificare` **pe jumătate**: doar câmpul `readonly fragment: string` și constructorul cu `super("Contine")`. Rulează `npx tsc --noEmit` și copiază mesajul în răspuns.
- Completează `trece`: `true` dacă valoarea conține fragmentul. Pune `new Contine("-")` la finalul array-ului.
- Scrie în `main` linia `const v = new Verificare("x");`, rulează `tsc`, copiază mesajul, apoi șterge linia.

leșire așteptată: fiecare linie capătă la final `Contine: OK` pentru `ESID-100` și `Contine: FAIL` pentru celelalte două. Restul e identic.

Teorie: `C2-polimorfism-abstract` — `abstract corp(): string;` în `Notificare` și cele două erori din `README` (subclasă fără `corp`, `new` pe abstractă).

Indiciu: cele două mesaje pe care trebuie să le vezi încep cu `Non-abstract class 'Contine' does not implement` și `Cannot create an instance of an abstract class`. Dacă `tsc` tace la pasul 3, n-ai pus `abstract` pe metodă.

Gata când: ambele mesaje sunt în răspuns, `Contine` apare în ieșire, iar `tsc` e curat la final.

De reținut: un corp care există doar ca să treacă compilatorul e semnul metodei abstracte. Acum compilatorul nu te lasă să termini pe jumătate.

2. `text()` scris o singură dată

nivel 1

Fișierul `03-raport/main.ts`. Acum `RaportSemnat` și `RaportCuData` rescriu fiecare `text()`. Reorganizează:

- `abstract class Raport`, cu două metode abstracte: `abstract antet(): string;` și `abstract subsol(): string;`.
- `text()` rămâne doar în `Raport` și se compune din: `antet()`, titlul, liniile, `subsol()` — sărind peste bucățile goale, ca să nu iasă linii goale.
- `RaportCuData`: șterge `text()`; scrie `antet()` care întoarce `Generat: ...` și `subsol()` care întoarce `""`.
- `RaportSemnat`: șterge `text()`; `antet()` întoarce `""`, `subsol()` întoarce `-- autor`.
- `new Raport("Simplu")` din exercițiul 4 al fișei de polimorfism nu mai compilează. Scrie `class RaportSimplu extends Raport` cu ambele metode întorcând `""` și folosește-l acolo.

Ieșire așteptată: identică cu cea de dinainte, pe tot fișierul, inclusiv `RaportCompus`.

Teorie: C2 — `trimite()` e scris o dată în `Notificare` și cheamă `corp()`, pe care încă nu-l cunoaște.

Indiciu: compunerea în bază:

```
const bucati = [this.antet(), this.titlu, ...this.linii, this.subsol()];
return bucati.filter((b) => b !== "").join("\n");
```

`filter` e ce scoate linia goală când antetul sau subsolul e `""`.

Gata când: `text()` apare o singură dată în fișier (`grep -c "text()"` pe definiții) și ieșirea e neschimbată.

De reținut: metoda scrisă din bază cheamă metode nescrise încă. Baza fixează ordinea bucăților; subclasa dă bucata.

3. Regula rândului, ca metodă abstractă

mai greu

Fișierul `01-set-de-date/main.ts`. `SetDeDateCuCheie` și `SetDeDateStrict` rescriu amândouă `adauga`. Reorganizează:

- `abstract class SetDeDate` cu `protected abstract verificaRand(rand: string[]): void`; — aruncă `Error` dacă rândul nu e bun, altfel nu face nimic.
- `adauga(rand)` rămâne scris o singură dată, în bază, în ordinea: verifică numărul de valori (ca acum), cheamă `this.verificaRand(rand)`, adaugă rândul.
- Cele două subclase: șterge `adauga`; mută condiția fiecăreia în `protected verificaRand(rand: string[]): void`.
- `new SetDeDate(...)` nu mai compilează. Scrie `class SetDeDateSimplu extends SetDeDate` cu un `verificaRand` care nu face nimic, și folosește-l unde era `new SetDeDate`.

Ieșire așteptată: identică cu cea de dinainte, cu o singură diferență posibilă: la `SetDeDateStrict`, rândul `["ESID-779", ""]` dă acum obligatoriu `Row has 2 values, expected 3`, pentru că ordinea o fixează baza. Scrie într-un rând dacă asta e ordinea pe care o alesiși la fișa de moștenire.

Teorie: C2 + exercițiul 2 de mai sus (aceiași tipar: baza fixează pașii).

Indiciu: corpul lui `verificaRand` e exact codul dinaintea lui `super.adauga(rand)` din vechiul `adauga`, fără apelul la `super`. Pentru `SetDeDateSimplu`, o metodă cu corp gol `protected verificaRand(): void {}` e corectă — nu are ce verifica.

Gata când: `adauga` apare o singură dată în fișier; toate mesajele de eroare apar la cazurile lor; `randuri`: dă aceleași numere.

De reținut: baza fixează *ordinea pașilor* (numără, verifică, adaugă); subclasa umple un singur pas. Nimeni nu mai poate uita să numere coloanele.

Partea 2 — un contract peste clase fără rudă comună

4. Exportabil

mai greu

Folder nou: `src/fise/exportabil/main.ts`. Copiază acolo `Raport` cu `RaportSemnat`, `SetDeDate` cu `SetDeDateSimplu`, și `Exportator` cu `ExportatorCsv` (și clasa cu separator, dacă `Csv` o folosește). Fără

codul lor de rulare.

1. `interface Exportabil { caRanduri(): string[][]; }.`
2. `abstract class Raport implements Exportabil: caRanduri()` întoarce fiecare linie din `text()` ca un rând cu o singură valoare.
3. `abstract class SetDeDate implements Exportabil: caRanduri()` întoarce rândurile.
4. `function exporta(lucruri: Exportabil[], e: Exportator): void` : pentru fiecare, afișează `e.umeFisier("export")` și `e.converteste(l.caRanduri())`.

În `main` :

```
const raport = new RaportSemnat("Rezultate", "andreea");
raport.adauga("3 teste verzi");

const set = new SetDeDateSimplu(["ESID", "STATUS"]);
set.adauga(["ESID-100", "Active"]);
set.adauga(["ESID-777", "Pending"]);

const lucruri: Exportabil[] = [raport, set];
exporta(lucruri, new ExportatorCsv());
```

Ieșire așteptată:

```
export.csv
Rezultate
3 teste verzi
-- andreea
export.csv
ESID-100,Active
ESID-777,Pending
```

Teorie: README-ul secțiunea C, plus `C1` : lista mixtă. Diferența: aici tipul comun e o interfață, nu o clasă.

Indiciu: pentru raport: `this.text().split("\n").map((linie) => [linie])` . O clasă poate avea `extends X implements Y` în aceeași linie, iar o clasă abstractă poate implementa o interfață cu o metodă concretă.

Gata când: `lucruri` e declarat o singură dată și ține obiecte din ambele ierarhii; ies cele șapte linii; în `exporta` nu apare `instanceof` .

De reținut: cu `extends` n-ai fi putut face lista asta. Un set de date nu *este* un raport; doar *poate* același lucru.

5. Contractul scris de altcineva

mai greu

Fișierele `03-raport/main.ts` și `01-set-de-date/main.ts` . Limbajul are două contracte gata scrise: ``${obiect}`` cheamă `toString()` , iar `JSON.stringify(obiect)` cheamă `toJSON()` dacă există.

1. Întâi, fără nicio modificare, adaugă în `main` -ul fiecărui fișier: `console.log(`${raport}`)` respectiv `console.log(JSON.stringify(set))` . Rulează și copiază cele două ieșiri în răspuns.
2. În `Raport` : `toString(): string` care întoarce `this.text()` .
3. În `SetDeDate` : `toJSON()` care întoarce obiectul `{ coloane: this.coloane, numarRanduri: this.numarRanduri() }` .
4. Rulează din nou aceleași două linii.

Ieșire așteptată la pasul 4:

```
Rezultate suita
3 teste verzi
1 test rosu
-- andreea
{"coloane": ["ESID", "FR_GUID", "STATUS"], "numarRanduri": 3}
```

Teorie: nu e un demo din `src/teorie/`; e contractul limbajului. Caută în documentația TypeScript/JavaScript „toJSON”.

Indiciu: la pasul 1 trebuie să vezi `[object Object]` și un JSON care conține **toate** rândurile, deși `randuri` e `private`: `private` e regulă de compilare, nu de rulare. Numele `toString` și `toJSON` trebuie scrise exact așa; nimeni nu le cheamă explicit.

Gata când: ambele ieșiri „înainte” sunt în răspuns, iar cele „după” sunt cele de mai sus.

De reținut: aici n-ai inventat tu contractul. L-ai respectat pe al limbajului ca să-i poți folosi codul. Așa lucrezi cu orice bibliotecă, Playwright inclus.

Partea 3 — provocări

6. Un intrus în listă

provocare

Folderul `src/fise/exportabil/`. Scrie:

- `class Notita implements Exportabil`, câmpuri `readonly titlu: string` și `readonly text: string`, constructor cu amândouă. Nu moștenește nimic.
- `caRanduri()` întoarce două rânduri: `[titlu]` și `[text]`.
- Adaugă `new Notita("De verificat", "ESID-78 are 7 caractere")` la finalul lui `lucruri`.

Ieșire așteptată: cele șapte linii de la 4, apoi:

```
export.csv
De verificat
ESID-78 are 7 caractere
```

Apoi `git diff src/fise/exportabil/main.ts`.

Teorie: exercițiul 4 de mai sus; aceeași interfață, un al treilea tip.

Indiciu: `Notita` are doar `implements`, fără `extends`. Dacă `tsc` se plânge, cel mai probabil numele metodei nu e exact `caRanduri` sau tipul întors nu e `string[][]`.

Gata când: ies cele trei linii noi, iar în `git diff` funcția `exporta` nu apare.

De reținut: asta e testul adevărat al unei interfețe: un tip la care nu te-ai gândit când ai scris-o întră fără să schimbi nimic.

7. Verificare : clasă abstractă sau interfață?

provocare

Fișierul `05-verificari/main.ts`. Răspunde în scris, în cinci-șase rânduri, pornind de la două întrebări:

- `nume` e un câmp pe care îl au toate verificările. Dacă `Verificare` ar fi interfață, cine ar ține câmpul și constructorul care îl setează?
- `Negata` face `super("Nu" + interior.nume)`. Ce devine linia asta dacă nu mai există clasă de bază?

Apoi implementează varianta pe care ai ales-o și rulează tot fișierul, inclusiv `careNuTrec` și `Negata`.

leșire așteptată: identică cu cea de dinainte, oricare ar fi varianta.

Teorie: README secțiunea C2 și exercițiile 1 și 4 de pe fișa asta, puse față în față.

Indiciu: dacă alegi interfața, `nume` trebuie declarat și setat în fiecare dintre cele șapte clase; numără liniile dublate cu `git diff --stat` și pune numărul în motivare. Dacă rămâi la clasa abstractă, nimic nu se mută: și asta e un argument, dacă îl scrii.

Gata când: programul afișează la fel ca înainte, iar motivarea se potrivește cu ce a trebuit să muți sau să dublezi în cod.

De reținut: cele două nu se exclud. Cea mai obișnuită soluție e o clasă abstractă plus interfețe: familia vine din moștenire, capacitățile din contracte.